

RAG-Augmented Multi-Agent LLM Framework for Explainable Software Vulnerability Detection Using CWE Knowledge Bases

Richa Verma (25MCSS02) Advisor: Dr. Akshay Pandey

One shape for every answer

Where this fits

We can mark risky lines, and we can search the CWE book. What was missing was a single shape for the written answer. Without that, two runs cannot be compared.

This note describes that form. It does not add a new detection score.

The idea we started with

Two systems can only be compared if they answer in the same shape. If one system returns a paragraph and another returns a JSON object with different fields, a table of scores is hiding a change in the question.

The form is also a safety boundary. If the answer may contain any extra text, a model can drift into writing an attack. A closed form with no payload field makes that drift fail the check.

Why we chose this way

We required a decision, a CWE id and name, cited lines, a cause, an explanation, and a fix. We allowed three decisions: unsafe, safe, and not sure. Not sure exists so the system can abstain instead of guessing.

Line numbers start at 1 because that is how a person counts lines in an editor. A citation that cannot be found in the file is a failed answer, even if the decision happens to be right.

We check the form with a schema, which is a machine-readable contract, plus a few extra rules: the CWE must be in our book, and the cited snippet must occur in the file. We do not require the decision to agree with the pattern checker. Agreement is not the same as truth.

What we actually did

Every answer must fill the same fields. If a required field is missing, or if the text is not valid JSON, the answer is rejected.

The decision is one of three words: unsafe, safe, or not sure. Not sure is allowed. We would rather see an honest abstention than a guessed yes.

The CWE field must be an id and a name. The lines field must name the file and the line numbers, counted from 1, the same way an editor counts. The other fields are a short cause, a plain explanation, and a fix.

There is no field for attack steps. The tool is not asked to write an exploit or a payload.

We store two sample answers, one unsafe and one safe, and one broken sample. A small check accepts the good samples and rejects the broken one. That check does not need a live model.

The pattern checker still only marks lines. The form is the contract for whoever writes the final note, whether that writer is a template or a model.

A concrete example

A valid unsafe answer for the SQL example names the decision unsafe, the CWE as CWE-89, the lines that build the query, a cause such as "user text is joined into the command", a short explanation, and a fix that says to use a prepared query.

A valid safe answer for the paired file uses the same fields. The decision is safe. It can still name CWE-89, because that is the risk that was checked.

If the cited lines are blank, or the decision is a word we did not allow, the check fails. We keep that failure. We do not fill the hole by hand.

Picture

Every answer uses the same fields

Decision

unsafe / safe / not sure

CWE

id and name

Lines

file and line numbers

Why

one short cause

Explain

plain words

Fix

what to change

Every run has to fill these fields. A missing field makes the answer invalid.

The numbers

Field	What it means
Decision	Unsafe, safe, or not sure.
CWE	Id and name from the book.
Lines	File and line numbers, starting at 1.
Cause and explanation	Why the lines are risky, in plain words.
Fix	What a programmer should change.

The form is strict on purpose. Extra fields are not allowed, so the answer cannot hide an exploit in an extra box.

What we concluded, and why

The pieces now exist separately: files, a checker, a book, search, and a form.

The next piece of work is to run them in one order, still without calling a paid model. The conclusion of this note is a contract, not a score: every later system must fill this form, and a broken form is a failed run.

What we are not saying

A valid form is not a correct decision. The checker can still be wrong about whether the code is

unsafe.